PROJECT SPECIFICATION

Be My Sponsor

A self-serve marketplace for embeddable sponsorship slots — publishers list ad spots, sponsors book them, a widget serves them.

Document Product & Engineering Specification v1.0
Status Draft — for build (Phase 1: Walking Skeleton)
Owner swarnilsinghaicse@gmail.com
Date 24 July 2026
Base nextjs/saas-starter → monorepo (pnpm + Turborepo)

Contents

1 Product Overview

Be My Sponsor lets any website owner turn unused surface area (a sidebar, a footer, a newsletter block) into a bookable sponsorship slot, and lets sponsors discover and book those slots self-serve — without ad networks, tracking cookies, or an ad-ops team.

THE PROBLEM

Small publishers have valuable audiences but no lightweight way to sell sponsorship directly. Sponsors have budget but no easy way to find and book independent inventory. Existing ad networks are heavy, opaque, and privacy-hostile.

THE APPROACH

A publisher defines a **Property** (their site) and one or more **Slots** (fixed-size ad spots). They drop a tiny embeddable **widget** on their page. A sponsor **books** a slot for a date range; the **ad-server** serves the booked creative; empty slots show a "Sponsor this spot" call-to-action.

Why it can work: zero layout shift (dimensions reserved), no third-party cookies, a <15 KB widget, and a booking model that makes double-booking structurally impossible via a database-level overlap constraint.

2 Personas & Core Concepts

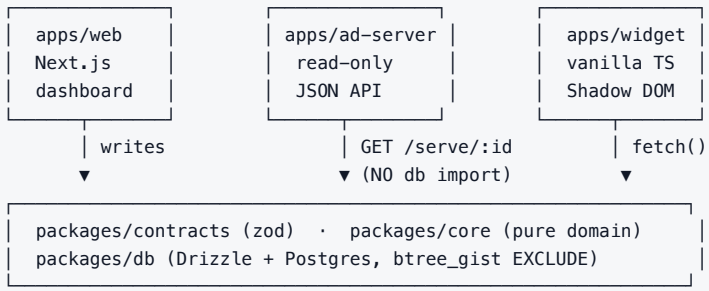
Persona	Goal	Key actions
Publisher	Monetise their site with direct sponsorship	Create Property → create Slots → embed widget → approve/serve bookings
Sponsor	Reach a specific audience for a date range	Discover a slot → book a date range → upload creative → pay LATER
Both	Publisher on one site, sponsor on another	Belongs to multiple orgs, switches between them

DOMAIN VOCABULARY

Term	Meaning
Organization	A billing/ownership unit, typed `PUBLISHER`, `SPONSOR`, or `BOTH`. All inventory and bookings belong to an org, never a bare user.
Membership	A user→org link carrying a role and a permissions array. A user may hold memberships in many orgs simultaneously.
Property	A website or surface owned by a publisher org (has a public slug).
Slot	A fixed-dimension ad spot on a property (Phase 1: sidebar image, `300×250` only). Has a public id used by the widget/ad-server.
Booking	A sponsor org reserving a slot for a date range. Governed entirely by the booking state machine.
Creative	The image a sponsor supplies for a booking (hardcoded in Phase 1).

3 System Architecture

A pnpm-workspace monorepo orchestrated by Turborepo. Domain logic is isolated from frameworks: `packages/core` is pure TypeScript with zero imports from Next, React, or Drizzle. Anything crossing an HTTP boundary is validated by a Zod schema from `packages/contracts`.



Unit	Responsibility	Constraint
`apps/web`	Auth, dashboard, publisher/sponsor UI	Next.js App Router, Better Auth (org plugin)
`apps/ad-server`	Serve creative JSON for a slot	Standalone, read-only, **must not import packages/db**
`apps/widget`	Embed & render a slot on publisher pages	Vanilla TS, esbuild, no deps, ≤15 KB gzipped
`packages/db`	Schema, migrations, DB access	Drizzle + Postgres; money = integer minor units
`packages/contracts`	Zod schemas for every HTTP boundary	Single source of wire-format truth
`packages/core`	Framework-free domain logic	Pure; zero next/react/drizzle imports

4 Repository Layout

```
be-my-sponsor/
├─ apps/
│  ├─ web/            # Next.js App Router + Tailwind + shadcn/ui
│  ├─ ad-server/      # read-only JSON API (no db import)
│  └─ widget/         # vanilla TS, esbuild, Shadow DOM, ≤15KB gz
├─ packages/
│  ├─ db/             # Drizzle schema + raw SQL migrations
│  ├─ contracts/      # zod schemas (HTTP boundary types)
│  └─ core/           # pure domain: transition() + unit tests
├─ CLAUDE.md          # binding product spec (rules for all code)
├─ TODO.md            # phase checklist, updated after each step
├─ turbo.json         # Turborepo pipeline
├─ pnpm-workspace.yaml
├─ tsconfig.base.json # shared strict TS config
├─ .env.example
└─ package.json       # workspace root
```

> **NOTE**
> The build starts from `nextjs/saas-starter` (a single Next app), which is progressively restructured into the monorepo above. The starter's Stripe wiring is removed for Phase 1.

5 Data Model PHASE 1

Drizzle schema in `packages/db`. All monetary values are stored as **integer minor units** (e.g. cents) with a companion `currency` column — **no floating-point money anywhere**. Every migration is reversible and additive.

5.1 Tables

Table	Key columns	Notes
user	id · email · name · createdAt	Managed largely by Better Auth.

organization	id · name · slug · **type** (`PUBLISHER` | `SPONSOR` | `BOTH`)	Owns all inventory & bookings.
membership	id · userId → user · orgId → organization · role · **permissions[]**	Many-to-many; a user may join multiple orgs.
property	id · orgId → organization · name · **slug** · createdAt	Belongs to a publisher org.
slot	id · propertyId → property · **publicId** · name · width · height · placement	Phase 1: width=300, height=250, placement= `sidebar` .
booking	id · slotId → slot · sponsorOrgId → organization · dateRange · **status** (enum) · priceMinor · currency	Minimal in Phase 1: status enum + overlap guard.

5.2 The overlap constraint CRITICAL

A slot must never hold two active bookings whose date ranges overlap. This is enforced **in the database**, not application code, via a raw SQL migration that enables `btree_gist` and adds an `EXCLUDE` constraint:

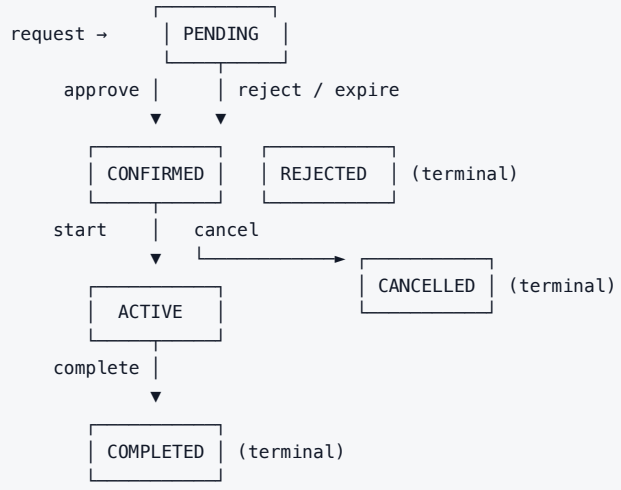
```
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE booking ADD CONSTRAINT booking_no_overlap
  EXCLUDE USING gist (
    slot_id      WITH =,
    date_range   WITH &&
  )
  WHERE (status IN ('confirmed', 'active'));   -- only live bookings block
```

> **WHY DATABASE-LEVEL**
>
> Application checks race under concurrency. An `EXCLUDE` constraint makes double-booking a rejected transaction, not a bug you can forget to write.

6 Booking State Machine PHASE 1

The booking lifecycle lives in `packages/core` as a **pure** function `transition(booking, event)` with an exhaustive `switch` . A booking's status is **never** written directly — only ever through `transition()` . Invalid transitions throw.



From	Event	To
PENDING	APPROVE	CONFIRMED
PENDING	REJECT / EXPIRE	REJECTED
CONFIRMED	START	ACTIVE
CONFIRMED	CANCEL	CANCELLED
ACTIVE	COMPLETE	COMPLETED

REJECTED · CANCELLED · COMPLETED are terminal — any event throws.

DESIGN CONTRACT

`transition()` takes framework-free plain data, returns the next booking (or throws), and is covered by unit tests. It is the **only** writer of booking status across the entire system.

7 Tech Stack & Dependencies

FOUNDATION

- TypeScript (strict mode, shared base config)
- pnpm workspaces + Turborepo
- ESLint + Prettier
- PostgreSQL + Drizzle ORM

APPS

- **web:** Next.js App Router, Tailwind, shadcn/ui
- **auth:** Better Auth + organization plugin
- **widget:** vanilla TS, esbuild, no dependencies
- **validation:** Zod (contracts)

DEPENDENCY DISCIPLINE

No dependency may be added unless it is named in **CLAUDE.md §8**. Anything outside that list requires explicit approval before install.

8 Engineering Rules BINDING

These are non-negotiable and apply to **all** code written for the project. They mirror `CLAUDE.md`, which is the source of truth.

#	Rule
R1	A user must be able to belong to **multiple organizations** and switch between them (CLAUDE.md §2). Non-negotiable.
R2	Never write a booking status directly — **only** via `transition()`.
R3	No domain logic in route handlers. Handlers **validate → authorize → call core → serialize**. Nothing else.
R4	Everything crossing an HTTP boundary has a Zod schema in `packages/contracts`.
R5	`packages/core` imports nothing from next / react / drizzle.
R6	`apps/ad-server` must not import `packages/db`.
R7	Money = integer minor units + currency column. No floats anywhere.
R8	Every migration is reversible and additive.
R9	No dependency outside CLAUDE.md §8 without approval.
R10	Widget build fails above **15 KB gzipped** (CI-checked size budget).
R11	Widget reserves exact dimensions — **zero layout shift**.
R12	Commit after each numbered step with a conventional-commit message; update `TODO.md` after each step.
R13	If a step is bigger than expected, stop and report rather than half-finish.

9 Phased Roadmap

Phase	Theme	Delivers
1	**Walking Skeleton**	Monorepo, schema, contracts, state machine, auth + multi-org, create Property/Slot, embed snippet, read-only ad-server, embeddable widget with CTA. End-to-end thin slice, no money.
2	Booking flow	Sponsor-facing booking of a date range, creative upload, publisher approval, driving the state machine end to end.
3	Payments	Stripe integration, priced slots, invoices — money moves.
4	Calendar & availability	Availability calendar UI, scheduling, live serving by date.
5	Stats & reporting	Impressions/clicks, sponsor & publisher dashboards.

> **SCOPE FENCE**
> Phase 1 builds **none** of Phase 2+. No payments, no booking flow, no calendar, no stats. Writing Stripe code in Phase 1 is out of scope — stop.

10 Phase 1 Build Plan WALKING SKELETON

Planning first: before any file is written, produce the exact folder/file tree, the proposed Drizzle schema table-by-table, any spec ambiguities, and anything to defer — then wait for approval. After approval, scaffold in order, committing after each step.

Step	Deliverable	Commit
1	Monorepo skeleton: pnpm workspaces + Turborepo, TS strict, shared `tsconfig.base`, ESLint + Prettier, `.gitignore`, `.env.example`, root `TODO.md`.	`chore: scaffold monorepo`
2	`packages/db`: Drizzle schema + migrations for Phase 1 entities. Raw SQL migration enabling `btree_gist` + `EXCLUDE` overlap constraint. Money as integer minor units.	`feat(db): phase-1 schema`
3	`packages/contracts`: Zod schemas for all entities crossing the HTTP boundary.	`feat(contracts): zod schemas`
4	`packages/core`: pure `transition(booking, event)` with exhaustive switch + unit tests. Nothing else.	`feat(core): booking state machine`
5	`apps/web`: Next.js + Tailwind + shadcn/ui. Better Auth org plugin (multi-org switch). Screens: sign up, sign in, org switcher, create Property, create Slot (300×250 sidebar), embed-snippet page.	`feat(web): auth + dashboard`
6	`apps/ad-server`: standalone read-only. `GET /serve/:slotPublicId → {creative|null, bookingId|null, width, height}`. Hardcoded creative allowed. No `packages/db` import.	`feat(ad-server): serve endpoint`
7	`apps/widget`: vanilla TS + esbuild, Shadow DOM, reserves exact dimensions, fetches ad-server; no creative → "Sponsor this spot" CTA → `/{property-slug}/sponsor?slot=…`. CI size budget ≤15 KB gz.	`feat(widget): embeddable slot`

AD-SERVER RESPONSE CONTRACT

```
GET /serve/:slotPublicId  →  200 application/json
{
  "creative":  { "imageUrl": string, "clickUrl": string } | null,
  "bookingId": string | null,
  "width":     300,
  "height":    250
}
```

11 Out of Scope — Phase 1

DO NOT BUILD

- Stripe / payments of any kind
- The sponsor booking flow
- Availability calendar
- Impression / click stats
- Creative upload pipeline (hardcoded in P1)
- Multiple slot sizes / placements

DEFERRED (DESIGN LATER)

- Pricing model & currency handling UX
- Notifications / email
- Public marketplace / discovery
- Fraud & content moderation of creatives
- Rate limiting on ad-server

12 Acceptance Criteria — Phase 1 Done

1. A user can sign up, sign in, and belongs to at least one org; can create a second org and **switch** between them.
2. A publisher org can create a Property and a 300×250 sidebar Slot, and view its embed snippet.
3. The DB rejects two overlapping active bookings on the same slot (constraint verified).
4. `transition()` passes its unit tests and is the only writer of booking status.
5. `ad-server` returns valid JSON for a slot public id and imports nothing from `packages/db`.
6. The widget renders into Shadow DOM with zero layout shift and shows the CTA when there is no creative; build is ≤15 KB gzipped.
7. `TODO.md` reflects every completed step; each step is a conventional commit.